Queremos un sistema que pueda **fabricar distintos tipos de misiles**:

- **Misil antiaéreo**
- **Misil antitanque**

Cada misil requiere una **construcción compleja** (tipo,guiado, ojiva, alcance y propulsion), por lo que usaremos el **patrón Builder** para ensamblarlos paso a paso. Además, queremos que la **creación del tipo de misil** esté **desacoplada**, así que aplicamos el **Factory Method** para decidir qué *builder concreto* usar.

SOLUCIÓN:

// === Producto complejo ===

```
class Misil {
    private String tipo;
    private String guiado;
    private String ojiva;
    private String propulsion;
    private int alcance;

    public void setTipo(String tipo) { this.tipo = tipo; }
    public void setGuiado(String guiado) { this.guiado = guiado; }
    public void setOjiva(String ojiva) { this.ojiva = ojiva; }
    public void setPropulsion(String propulsion) { this.propulsion =
propulsion; }
    public void setAlcance(int alcance) { this.alcance = alcance; }

    @Override
    public String toString() {
        return "Misil [" + tipo + "] → " +
               "Guiado: " + guiado +
               ", Ojiva: " + ojiva +
               ", Propulsion: " + propulsion +
               ", Alcance: " + alcance + " km";
    }
}

// === Builder ===
abstract class MisilBuilder {
```

```
    /**********************/
}

// === Factory Method ===
abstract class MisilFactory {
    public abstract Misil createMisil(); // ← Factory Method
}

class AntiAirMisilFactory extends MisilFactory {
    /****************/
}

class AntiTankMisilFactory extends MisilFactory {
    /****************/
}

// === Cliente ===
public class DefenseSystem {
    public static void main(String[] args) {

        MisilFactory airFactory = new AntiAirMisilFactory();

        System.out.println(airFactory.createMisil());

        MisilFactory tankFactory = new AntiTankMisilFactory();
        System.out.println(airFactory.createMisil());
    }
}
```